Eval Incarnate: A Builder's Curriculum

From Sitcom Reruns to Living Worlds

Dnf_Jeff

2026

Foreword

Learning to Judge, Gently

Most people come to systems like these looking for answers.

How do I make the AI smarter? How do I make the world feel alive? How do I stop things from going off the rails?

This curriculum is not interested in those questions — at least not directly.

It starts somewhere quieter, and harder to admit:

How do we decide what "good" looks like? Who gets to judge? By what criteria? And what happens when those criteria are wrong — or incomplete — or humane in one context and cruel in another?

For a long time, computing pretended these questions didn't exist. We built simulations that looked neutral, algorithms that claimed objectivity, and systems whose most important assumptions were buried in code no one could see. We learned how traffic flows, how markets oscillate, how needs get satisfied — but not how values are chosen.

This work is about turning that inside out.

It treats judgment not as a bug to eliminate, but as a first-class mechanic. It treats bias not as something to hide, but as something to declare, inspect, and revise. It treats the language model not as an oracle, but as an interpreter — a medium that runs the programs we give it, whether we acknowledge that responsibility or not.

So yes, along the way, you'll build rooms and characters and objects. You'll write YAML. You'll watch a filesystem turn into a town. You'll see gossip propagate, reputations form, conflicts resolve (or not). It will feel like game development.

That's intentional.

Games are one of the few cultural spaces where people are allowed to experiment with rules, fail safely, and ask "what if the world worked differently?" without needing permission. They give us something to do with our hands while our understanding catches up.

But the real lesson here isn't about games.

It's about evaluation — how it shapes behavior, how it distributes power, how it

quietly governs everything from families to cities to algorithms. It's about learning to see the rules that are already running, and discovering that once you can see them, you can change them.

If you read this looking for a framework to dominate, optimize, or control, you'll be disappointed.

If you read it looking for a way to build systems that are legible, corrigible, and kind without being naive — systems that can explain themselves, argue with themselves, and grow — you're in the right place.

This curriculum doesn't promise certainty. It promises inspectability.

It doesn't promise neutrality. It promises honesty.

And it doesn't teach you what to judge. It teaches you how to notice that you already are.

Welcome to Mayberry. The porch light's on.

Eval Incarnate: A Builder's Curriculum

> *From sitcom reruns to living worlds — how language models become interpreters, filesystems become stages, and judgment becomes gameplay.*

What This Is

This is a self-contained curriculum that teaches you how to build **living systems** — worlds where characters think, objects advertise what they can do, rooms inherit rules from their neighborhoods, and the language model itself becomes the universal interpreter running it all.

It masquerades as game development. That's intentional. The game is the interface — a familiar, lighthearted way to learn mechanisms that are far deeper than any game. The real goal is understanding how to build systems where evaluation, agency, and meaning emerge from simple structures.

Think of it like this: Andy Taylor teaching Opie to fish isn't really about catching bass. It's about patience, observation, and responsibility. The fishing pole is just the interface.

How to Read This

Each Part is a self-contained unit with chapters. Read them in order the first time. After that, use the chapter index below to jump to whatever you need.

Part	Title	What You'll Learn
Part 1	**Welcome to Mayberry**	Why we're here, what we're building, and the key insight that changes everything
Part 2	**The Linguistic Motherboard**	How interpreters work, the Axis of Eval, skills as programs, and the architecture underneath
Part 3	**Building Living Worlds**	Rooms, objects, characters, navigation, persistence, and the Simulator Effect
Part 4	**The Eval Awakening**	SIM vs EVAL, judgment as mechanic, making hidden assumptions visible, ethics and safety
Part 5	**Speed of Light**	Multi-turn simulation, empathic expressions, K-lines, Play-Learn-Lift, and where this all goes

Chapter Index

Part 1: Welcome to Mayberry

Part 2: The Linguistic Motherboard

Part 3: Building Living Worlds

Part 4: The Eval Awakening

- Chapter 14: From SIM to EVAL — A new kind of simulation
- Chapter 15: The Evaluator Effect — When the player realizes they're the judge
- Chapter 16: Making the Black Box White — Inspectable rules, declared bias
- Chapter 17: Ethics and the Tribute Protocol — Simulating real people responsibly
- Chapter 18: Safety as Architecture — Ethical framing inheritance

Part 5: Speed of Light

- Chapter 19: The Carrier Pigeon Problem — Why multi-agent systems lose precision
- Chapter 20: Empathic Expressions — Letting the LLM understand intent
- Chapter 21: K-lines and Activation — Names as memory triggers
- Chapter 22: Play-Learn-Lift — The methodology underneath everything
- Chapter 23: Where This Goes — From curriculum to creation

Running Examples

Throughout this curriculum, we use three running examples to make concepts concrete:

Example	Source	What It Teaches
Mayberry	*The Andy Griffith Show*	How a small town's social fabric models evaluation, judgment, reputation, and gentle governance
The Williams House	*The Danny Thomas Show*	How a family navigating show business models identity, performance, authenticity, and the boundary between public and private
The Gezelligheid Grotto	Don Hopkins' MOOLLM adventure	The canonical example — a living pub with characters, games, cats, and a monkey philosopher

---

Prerequisites

- Curiosity
- Comfort reading YAML (we'll teach what you need)
- Basic familiarity with the idea that an AI can read and write text
- No programming experience required, but it helps

Lineage

This curriculum synthesizes ideas from:

Thinker	Contribution
Don Hopkins	MOOLLM, Pie Menus, NeWS, HyperLook, SimCity, The Sims
Will Wright	SimCity, The Sims, the Simulator Effect
Alan Kay	Smalltalk, "The computer is a medium"
Marvin Minsky	Society of Mind, K-lines
Seymour Papert	Constructionism, Logo, microworlds
John Warnock	PostScript, the "linguistic motherboard"
Ivan Sutherland	Sketchpad — where interactive computing began

Part 1: Welcome to Mayberry

> *"You know what I think? I think you're trying to learn me something."* — Opie Taylor

Chapter 1: The Fishing Pole

Why a Game — and Why It's Not Really a Game

Let's get something out of the way: we're going to talk about game development. We'll discuss rooms, characters, objects, inventories, and quests. If you've ever poked around a game engine — even if you just watched a tutorial — some of this will feel familiar.

But the game is the fishing pole.

In *The Andy Griffith Show*, Andy doesn't lecture Opie about ethics. He takes him fishing. Out there on the lake, while they're waiting for a bite, Andy tells a story. Maybe it's about a time he made a mistake when he was young. Opie listens, not because he's being taught, but because they're doing something together. The lesson lands because the context is real, the relationship is genuine, and the activity gives Opie something to do with his hands while his mind works.

That's what we're doing here. The "game" gives you something to do with your hands. The real curriculum is about:

- How to build systems where **meaning emerges** from structure
- How to create **agents that evaluate**, not just simulate
- How to make **hidden assumptions visible** and editable
- How to construct **living worlds** from text files and a language model
- How to design **safety systems** that feel like architecture, not afterthoughts

These are not game design problems. They're systems design problems, AI alignment problems, epistemology problems. But they're a lot easier to learn when you're building a pub where a monkey philosopher plays Fluxx with a family of cats.

The Mayberry Principle

Andy Griffith understood something deep about governance. Mayberry didn't work because Andy had a gun. It worked because he rarely used it. The social fabric of the town — reputation, relationships, shared meals, gossip at Floyd's barbershop — did most of the heavy lifting.

When Otis Campbell stumbled in drunk on a Saturday night, Andy didn't throw the book at him. He left the cell door unlocked. Otis let himself in, slept it off, and let himself out in the morning. The *system* worked because it understood its participants, respected their dignity, and operated through social consensus rather than rigid enforcement.

That's the kind of system we're building. Not one that forces compliance through hard rules, but one where:

- **Rooms** have constitutions that inhabitants inherit
- **Characters** have inner lives that drive behavior
- **Judgment** is visible, editable, and distributed
- **Safety** comes from architecture, not restrictions

Barney Fife is what happens when you build a system with rigid enforcement and no contextual intelligence. Andy Taylor is what happens when you build a system that understands nuance, evaluates situations, and responds with appropriate force.

We're building Andy Taylor systems.

Chapter 2: The Thread

40 Years of Ideas That Led Here

Nothing appears from nowhere. The system we're going to build — let's call it what it is, a living world engine powered by a language model — inherits from a specific lineage of ideas. Understanding where these ideas come from isn't just academic. It tells you *why* the design works.

Here's the short version:

```
1962: Sketchpad (Sutherland)      — Multiple views of the same thing
1968: NLS/Augment (Engelbart)     — Hypertext, augmenting human intellect
1970s: Smalltalk (Kay)            — Everything is an object, all the way
down
1980: Society of Mind (Minsky)    — Intelligence as a society of simple
```

```
agents
1980: Constructionism (Papert)    — You learn by building inspectable things
1984: PostScript (Warnock)        — Text IS graphics IS programs
1986: NeWS (Gosling)              — Send programs, not pixels
1987: Self (Ungar & Smith)        — Prototypes and delegation, no classes
needed
1987: HyperCard (Atkinson)        — Everyone is a reader AND a writer
1989: SimCity (Wright)             — Simulation as mental model
1989: TinyMUD / LambdaMOO         — Text-based worlds users can build
2000: The Sims (Wright/Hopkins)   — Objects advertise, characters choose
2025: MOOLLM (Hopkins)            — Skills are programs, LLM is eval()
```

You don't need to memorize this. But notice the pattern: each step made computing more **spatial**, more **linguistic**, more **empowering**. Each step moved from "the computer tells you what to do" toward "you and the computer build something together."

What We Inherit

From this lineage, we get specific, usable concepts:

From PostScript and NeWS: The insight that you can send a *program* to an interpreter rather than sending *data* to a renderer. This is the single most important idea in the entire curriculum, and we'll unpack it in Chapter 3.

From The Sims: Objects *advertise* what they can do. A refrigerator says "I can satisfy hunger." A bed says "I can satisfy energy." Characters see these advertisements and choose based on their needs. This is how CARD.yml works — more on this in Part 2.

From Minsky's Society of Mind: A name can reactivate an entire mental state. When you say "Mayberry," you don't just recall a town name — you recall warmth, simplicity, gentle humor, front porches, fishing at the lake. That's a K-line. Names are triggers that activate context. We'll use this heavily.

From Papert's Constructionism: You learn by building things you can inspect. Not by reading about how to build things. Not by watching someone else build things. By getting your hands in and making something, then looking at what you made, then making it better. Play, Learn, Lift.

From the MUD/MOO tradition: A world made of rooms connected by exits. Users can build new rooms, create objects, define behaviors. Simple text commands create complex worlds. This is the oldest and most tested model for interactive virtual spaces.

Danny Thomas and the Performance Boundary

The Danny Thomas Show (also called *Make Room for Daddy*) ran from 1953 to 1965. Danny Williams was a nightclub entertainer juggling career and family — show business and home life.

What made the show work was the constant tension between Danny's **public performance** (confident, polished, in control on stage) and his **private reality** (baffled by teenagers, outwitted by his wife, struggling with fatherhood). The show's architecture was built around the boundary between these two worlds.

That boundary — between performance and authenticity, public and private, the role you play and the person you are — is exactly what we'll be building into our systems. In the framework we're learning:

- **The stage** is a room with performance framing. Everything that happens there is understood as enacted, not documentary.
- **The home** is a room with private framing. What happens there is canonical — it's the real state.
- **Characters** can exist in both spaces, and the system tracks which framing applies.

Danny Williams walking off-stage and into his living room is a *framing transition*. The rules change. The audience relationship changes. The evaluation criteria change. Our systems model this explicitly.

Chapter 3: The Key Insight

Send Programs, Not Data

In 1984, Adobe released PostScript. Most people think of it as a printer language. It was much more than that.

Before PostScript, if you wanted to print a circle, your computer calculated every pixel and sent them to the printer. The computer did the thinking; the printer was a dumb bitmap renderer. Send data. Receive printout.

PostScript flipped this. Instead of sending pixels, your computer sent a *program*:

```
newpath
100 200 50 0 360 arc
fill
showpage
```

That's not data. That's instructions: "Create a path. Draw an arc at position (100,200) with radius 50, from 0 to 360 degrees. Fill it. Show the page." The *printer* ran this program. The printer had intelligence.

John Warnock, Adobe's co-founder, described PostScript as a **"linguistic motherboard"** with **slots for capability cards.** The first card they built was a graphics card. But the architecture could accept any card — because the interpreter was universal.

Why This Matters for Us

A language model is a linguistic motherboard.

When you interact with a chatbot the old-fashioned way — "answer my question, here's some data" — you're sending data to a renderer. You're doing the pre-PostScript thing. You're calculating the pixels.

When you send the LLM a *skill* — a set of instructions, a description of behavior, a protocol for how to handle situations — you're sending a program to an interpreter. The LLM evaluates it. The LLM has intelligence.

Old Way	New Way
Send data to renderer	Send program to interpreter
Computer does all the thinking	Interpreter has intelligence
Fixed output format	Flexible, contextual responses
Every case needs its own code	One interpreter handles everything

This is the key insight that unlocks everything else in this curriculum:

> **Skills are programs. The LLM is `eval()`. Empathy is the interface.**

Let's break that down:

- **Skills are programs:** A skill isn't documentation about how to do something. It's a program that the LLM *runs*. When you write a skill that says "evaluate the social dynamics of this room," the LLM executes that instruction in context.
- **The LLM is `eval()`:** In programming, `eval()` takes text and executes it as code. The LLM does the same thing with natural language. It takes your skill description — text — and executes it as behavior.
- **Empathy is the interface:** The LLM doesn't need rigid syntax. It understands intent. You can say "make it warmer" or "increase the coziness" or "this room should feel like Floyd's barbershop on a Saturday morning" and it knows what you mean. That empathic understanding IS the interface.

The Mayberry Connection

Think of Andy Taylor as the interpreter. The townspeople bring him their problems — not as formal legal briefs, but as messy human situations. "Andy, Aunt Bee's feelings are hurt because Clara thinks her pickles are too sweet."

Andy doesn't parse this through rigid rules. He *evaluates* the situation: Who's involved? What are their needs? What's the history? What solution respects everyone's dignity? Then he acts.

That's exactly how a skill-based LLM system works:

1. The situation arrives (in YAML, Markdown, or plain English)
2. The interpreter (LLM) evaluates it against loaded skills
3. The response respects context, history, and relationships

4. The result is written back as state (files) or narrative (prose)

Andy Taylor is `eval()`. Mayberry is the filesystem. The town's customs are the skills.

Chapter 4: The Two Computers

How Sparse Rules Create Rich Worlds

Will Wright, creator of SimCity and The Sims, had a profound insight about simulation:

> *"He designs games to run on two computers at once: the electronic one on the player's desk, running his shallow tame simulation, and the biological one in the player's head, running their deep wild imagination."*

SimCity doesn't actually simulate a real city. It simulates *just enough* — some traffic patterns, some land values, some crime rates — and your brain fills in the entire rest of the city. You imagine the people, the smells, the politics, the Saturday farmers' market. The game gives you scaffolding. Your imagination builds the cathedral.

This is the **Simulator Effect**: players imagine simulations are vastly more detailed than they actually are.

The Two Computers in Practice

Here's what this looks like in our systems. Consider a character defined in YAML:

```yaml
character:
  name: "Barney"
  role: deputy
  personality: [nervous, eager, loyal, officious]
  carrying: one_bullet
  # Means well but overcomplicates everything
  # Quotes regulations nobody's heard of
  # Andy is his best friend and he'd do anything for him
```

That's Computer 1: the electronic one. Sparse data. A name, some traits, a few comments.

Now here's what happens when the LLM reads this and generates a scene:

> Barney straightened his holster for the third time, even though the single bullet was safe in his shirt pocket where Andy made him keep it. "Section 4, paragraph 2, subsection B of the Mayberry Municipal Code clearly states —"
>
> "Barney," Andy said, not looking up from the newspaper.

> "Right, right." Barney deflated slightly, then puffed back up. "But I'm just saying, Andy, if we don't enforce the jaywalking ordinance on a Tuesday, what's next? Anarchy?"
>
> Andy folded the paper. "Why don't we go get some lunch."
>
> Barney's face lit up like he'd been deputized all over again. "The diner? Your treat?"

That's Computer 2: the biological one (well, the neural one — the LLM serving as an imagination engine). From six lines of YAML, we get a scene that *feels* like those characters. The sparse data anchored it; the interpreter filled it in.

Scott McCloud's Masking

This connects to a principle from comics theory by Scott McCloud (*Understanding Comics*, 1993). McCloud observed that the most effective comics use **detailed, realistic backgrounds** but **simple, abstract characters:**

Element	Style	Effect
Environment	Detailed, specific	Immersive — you feel *there*
Characters	Abstract, simple	Projective — you see *yourself*

The Sims did this deliberately. The world was rendered in detail — furniture, textures, lighting. But the characters were simplified, spoke gibberish (Simlish), and were visually stylized. Players projected their own families, friends, and emotions onto those simple figures.

Our systems do the same thing with prose:

Component	Detail Level	Why
Rooms (ROOM.yml)	Rich — atmosphere, objects, exits, rules	You need to feel the space
Characters (CHARACTER.yml)	Sparse — traits, a few comments	The LLM projects personality

The LLM is the player's imagination, running on Computer 2.

The Danny Thomas Application

The Danny Thomas Show's apartment in New York is detailed: the specific furniture, the layout, the kitchen where Danny spills things. But the characters — Danny, Kathy, Rusty, Linda, Uncle Tonoose — are drawn with broad, memorable strokes. Danny is the blustering entertainer with a heart of gold. Uncle Tonoose is the impossible Lebanese relative. You don't need 50 pages of backstory. You need three traits and a catch phrase, and the audience's imagination does the rest.

When we build characters for our systems, we follow the same principle:

```yaml
character:
  name: "Uncle Tonoose"
  traits: [dramatic, opinionated, generous, impossible]
  catchphrase: "I am MORTIFIED!"
  # Shows up unannounced
  # Gives terrible advice with total confidence
  # Would take a bullet for family
  # His stories about Lebanon get longer every time
```

That's enough. The LLM — or better yet, the reader's imagination working with the LLM — fills in the rest. The Simulator Effect makes Uncle Tonoose feel like a real person from a few lines of YAML.

What We've Learned So Far

Let's take stock before moving on:

1. **The game is the fishing pole** — a familiar context for learning deeper mechanisms
2. **40 years of ideas converge** — PostScript, The Sims, Society of Mind, MOOs, and constructionism
3. **Send programs, not data** — the LLM is an interpreter, not a renderer
4. **Sparse rules, rich worlds** — the Simulator Effect lets minimal state create immersive experiences

In Part 2, we'll build the actual architecture — how the interpreter works, what skills look like, and how objects announce their capabilities.

Exercises

Exercise 1.1: Think of a location you know well — a barbershop, a diner, a porch, a living room. Write 5-7 lines of YAML describing it. Include an `atmosphere:` field and a few comments hinting at its history. Don't over-describe. Leave room for imagination.

Exercise 1.2: Pick a character from a show you love. Write a sparse CHARACTER.yml for them: name, 3-4 traits, one or two comments about their behavior. See how much personality you can convey in under 10 lines.

Exercise 1.3: Describe the difference between "sending data" and "sending a program" in your own words. Use a non-computing analogy — like the difference between giving someone a fish and teaching them to fish.

Part 2: The Linguistic Motherboard

> *"PostScript is a linguistic 'mother board', which has 'slots' for several 'cards'. The first card we built was a graphics card. We're considering other cards..."* — John Warnock, Adobe

---

Chapter 5: What Is an Interpreter?

From PostScript to LLMs

An interpreter is something that reads instructions and carries them out. That's it. Your microwave has one — you press buttons, it interprets them, it heats food. The interesting question is: how sophisticated is the interpreter?

Let's walk through three levels of interpreter sophistication, because this progression is exactly the history that leads to our systems.

Level 1: The Dumb Renderer

Before PostScript, printers were dumb renderers. Your computer did all the hard work — calculated every dot, every curve, every letter shape — and sent the printer a bitmap: "Put a dot here. Put a dot here. Put a dot here." Millions of dots.

The printer had no understanding of what it was printing. It couldn't tell a letter from a photograph from random noise. It just placed dots.

```
Computer: [calculates everything]
Computer → Printer: "Here are 8 million dots"
Printer: [places dots mechanically]
```

This is how most software still works. The application does all the thinking. The output channel is dumb.

Level 2: The Programmable Interpreter (PostScript)

PostScript changed the game. Instead of sending dots, you sent a program:

```
/Helvetica findfont 12 scalefont setfont
100 700 moveto
(Welcome to Mayberry) show
```

The printer reads this and *understands* it: "Find the Helvetica font, scale it to 12 points, move to position (100,700), and draw the text 'Welcome to Mayberry.'" The printer has intelligence. It knows about fonts, coordinates, curves, and fills.

The genius of PostScript was that text, graphics, and computation were all the same thing. A PostScript file is simultaneously:

- **Code** — it contains procedures, loops, conditionals
- **Data** — it describes structured information (fonts, positions)
- **Graphics** — it renders visual output

One language. Three dimensions. One interpreter.

John Warnock called this the **"linguistic motherboard"** — a universal interpreter with slots for different capability cards. The first card they plugged in was graphics. But the architecture wasn't limited to graphics. Any card would fit.

Level 3: The Universal Interpreter (LLM)

Now fast-forward forty years. A large language model is the next linguistic motherboard. Instead of understanding PostScript, it understands *everything you can express in language*:

```yaml
# This is simultaneously code, data, and graphics
character:
  name: Andy Taylor
  role: Sheriff of Mayberry
  approach: gentle_authority
  # Never leads with force
  # Asks questions instead of giving orders
  # Treats everyone with dignity, even Otis

  methods:
    - MEDIATE: resolve conflict between townspeople
    - COUNSEL: guide Opie through moral dilemma
    - DE_ESCALATE: calm down Barney's overreaction
```

When the LLM reads this, it understands Andy Taylor as a character. It can:

- **Execute** the methods (generate a mediation scene)
- **Query** the data (what's Andy's approach?)
- **Render** it visually (describe Andy walking down Main Street)

Same text. Three dimensions. One interpreter.

The LLM is the linguistic motherboard. Skills are the cards you plug into it.

The NeWS Connection

In 1986, James Gosling (who later created Java) built **NeWS** — the Network extensible Window System at Sun Microsystems. NeWS ran PostScript on the *server*. Your computer didn't send pixels or even drawing commands — it sent entire programs across the network. The server ran them.

This was radical. Instead of the old model:

```
Client does everything → sends pixels → Server displays them
```

NeWS did:

```
Client sends program → Server understands and executes it → Results appear
```

The server had intelligence. It could handle events, manage windows, run anima-

tions — all from programs sent as text over the wire.

This is exactly what we do with LLMs. You send a skill (program) to the LLM (interpreter). The LLM understands it and executes it. Results come back as text.

NeWS (1986)	Our System (2025)
Send PostScript program to server	Send skill description to LLM
Server interprets and executes	LLM interprets and executes
Server manages windows, events	LLM manages characters, narrative
Programs sent as text over network	Skills sent as text in context
Server has intelligence	LLM has intelligence

The technology changed. The architecture didn't.

Chapter 6: The Axis of Eval

Code, Data, and Graphics Unified

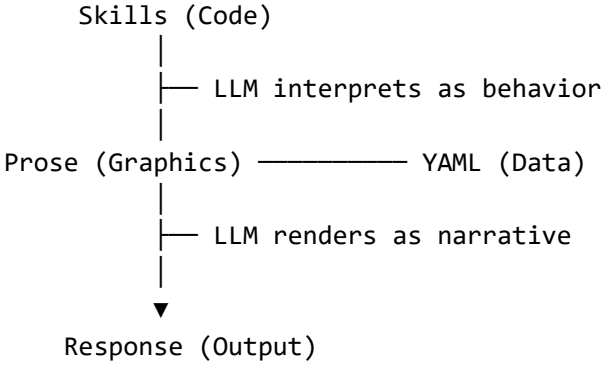
Don Hopkins coined the phrase **"Axis of Eval"** to describe HyperLook's unification of three dimensions. In PostScript and NeWS, the same text could be:

- **Code** — procedures to execute
- **Data** — structures to query
- **Graphics** — visuals to render

This wasn't a trick or a hack. It was fundamental. PostScript *is* text that *describes* graphics through *executable procedures*. The three dimensions aren't separate — they're three views of the same thing.

The Axis in Our Systems

In the systems we build, YAML and Markdown replace PostScript, and the LLM replaces the PostScript interpreter. But the axis holds:



Let's make this concrete. Here's a character file:

```yaml
character:
  name: "Danny Williams"
  profession: nightclub_entertainer
  location: home/living_room
  traits: [dramatic, loving, exasperated, generous]
  # His stage persona is confident; at home he's bewildered
  # Kathy runs the house; Danny just thinks he does
  # Uncle Tonoose's visits are natural disasters

  current_state:
    mood: frazzled
    reason: "Uncle Tonoose arriving tomorrow"
```

This single file is simultaneously three things:

Dimension	What the LLM Sees	Example
Data	Structured traits the LLM can query	`traits: [dramatic, loving, exasperated, generous]`
Code	Comments that instruct behavior	`# His stage persona is confident; at home he's bewildered`
Graphics	Descriptions that generate scenes	"Danny paced the living room, running his hand through his hair..."

The LLM **pivots** between these dimensions automatically. Ask it "what are Danny's traits?" and it reads data. Ask it "how would Danny react to Tonoose arriving early?" and it executes the behavioral code. Ask it "describe the scene" and it renders graphics in prose.

The Pivot Recipe

You can deliberately invoke different stances:

Stance	What You Ask	What the LLM Does
Data	"What's Danny's mood?"	Extract and report the YAML value
Code	"What does Danny do when Tonoose shows up?"	Interpret the comments as behavioral instructions
Graphics	"Describe the living room right now"	Generate a prose scene from the structured data

This isn't three different systems. It's one system, one file, three lenses. The Axis of Eval means you never have to choose. The same YAML file is your database, your script, and your scene description.

YAML Jazz: Comments as Intelligence

Here's something subtle but powerful. In most programming contexts, comments

are throwaway — notes for humans that the computer ignores. In our systems, comments are **semantic data**.

Look at this:

```yaml
room:
  name: "Floyd's Barbershop"
  type: commercial
  atmosphere: "warm, gossipy, slightly outdated"
  # The social nerve center of Mayberry
  # If you want to know anything, come here
  # Floyd tells you what you want to hear, not what's true
  # Two chairs but only one is ever used
  # The magazines are from 1958

  objects:
    barber_chair:
      status: occupied
      # Floyd is cutting hair and talking at the same time
      # He's not great at either one simultaneously
```

The LLM reads those comments. It understands them. When it generates a scene in Floyd's Barbershop, it knows that Floyd is unreliable, that the place is a gossip hub, that the magazines are ancient. The comments aren't decoration — they're instructions for the interpreter.

This is **YAML Jazz** — using the comment channel to convey meaning, emotion, behavioral guidance, and narrative voice. The YAML structure gives you queryable data. The comments give you soul.

Input vs Output Formats

One more distinction that matters:

Format	Best For	Why
YAML	Representing and manipulating state	Comments carry meaning, structure is editable
Markdown	Narrative, documentation, prose	Human-readable, embeds structure naturally
HTML/SVG	Rendering visual output	Display-ready
JSON	Machine-to-machine exchange	No comments = less expressive, avoid for state

YAML and Markdown are **input/working formats** — you read them, write them, and the LLM manipulates them. HTML and JSON are **output formats** — generated for display or exchange. This matters because you want to work in formats that carry the most meaning (YAML Jazz) and generate into formats appropriate for the consumer.

Chapter 7: Skills as Programs

Why Skills Aren't Documentation

Here's the paradigm shift that everything else depends on. In most AI systems, a "skill" or "prompt" is documentation: "You are a helpful assistant. When the user asks about weather, provide forecasts." It's a description. A manual. A README.

In our approach, a skill is a **program the LLM executes.**

The difference is not philosophical. It's operational:

Documentation	Program
Describes what to do	IS what to do
Read once, then wing it	Executed step by step
Static	Can create, modify, and delete files
Stateless	Tracks and persists state
One-size-fits-all	Instantiated with specific parameters

The Incarnation Spectrum

Skills exist on a spectrum of "aliveness":

Level	Form	Persistence	Example
Mentioned	Name invoked in conversation	Gone when chat ends	"Be polite, like Andy Taylor would"
Modeled	Behavior enacted in session	Lives in context window	Acting as a mediator during a conversation
Embedded	YAML data island in narrative	Lives in a document	Character traits stored in a story file
Incarnate	Directory with state files	Lives on disk forever	A full character with history, personality, relationships

The first level is what most people do with AI. The fourth level is what we're building toward.

An **incarnate** skill has:

1. **A CARD.yml** — machine-readable interface defining what it can do
2. **A README.md** — human-readable landing page
3. **State files** — YAML files that persist across sessions
4. **A home directory** — its place in the filesystem
5. **Inheritance** — properties it gets from parent directories
6. **A K-line** — its name activates everything associated with it

Traditional vs Incarnate: Side by Side

Let's compare. Here's a traditional "bartender" prompt:

```
You are a bartender. Be friendly and knowledgeable about drinks.
Recommend cocktails based on customer preferences.
Be responsible about alcohol consumption.
```

Here's the same concept as an incarnate skill:

```
skills/bartender/
├── README.md        # What the bartender skill does
├── SKILL.md         # Full protocol with frontmatter
├── CARD.yml         # Interface definition
└── templates/
    └── menu.yml     # Drink menu template
```

Where `CARD.yml` contains:

```yaml
name: bartender
tier: gameplay

methods:
  - name: TAKE-ORDER
    description: Accept and process a drink order
    parameters:
      - name: customer
        type: string
        required: true
  - name: RECOMMEND
    description: Suggest drinks based on mood/preferences
  - name: CUT-OFF
    description: Responsibly decline to serve

advertisements:
  TAKE-ORDER:
    visibility: public
    satisfies: [thirst, social]
  RECOMMEND:
    visibility: public
    satisfies: [curiosity, social]
  CUT-OFF:
    visibility: staff_only
    satisfies: [safety]

state:
  instance_creates:
    - "tabs.yml" # Running drink tabs
    - "86ed-list.yml" # Items currently unavailable
```

The traditional prompt is a note to yourself. The incarnate skill is a *living system* — it advertises capabilities, tracks state, has a defined interface, and can be in-

stantiated into specific bartender characters.

The Mayberry Skill

Let's build a real skill to make this tangible. Imagine we're creating the **mediator** skill — Andy Taylor's core capability:

```yaml
# skills/mediator/CARD.yml
name: mediator
tier: gameplay
lineage: "Andy Taylor → conflict resolution"

methods:
  - name: LISTEN
    description: Hear both sides without judgment
  - name: REFRAME
    description: Restate the problem so both sides feel heard
  - name: SUGGEST
    description: Propose a solution that preserves dignity
  - name: DE-ESCALATE
    description: Calm an escalating situation

advertisements:
  LISTEN:
    visibility: public
    satisfies: [emotional_need, trust]
    trigger: "when two or more characters are in conflict"
  SUGGEST:
    visibility: public
    satisfies: [resolution, harmony]

dovetails_with:
  - needs # Characters have emotional needs
  - room # Mediation happens in a place
  - reputation # Resolution affects standing
```

This skill can be *instantiated* into any character who needs mediation ability. Andy Taylor gets it. But so could Miss Crump, or even Floyd (who'd be terrible at it, which is also interesting).

Chapter 8: CARD.yml and Advertisements

Objects That Tell You What They Can Do

One of the most elegant inventions in The Sims — the game that Don Hopkins helped build at Maxis — was the **advertisement system.**

In most games, the developer hardcodes what objects do. Fridge → food. Bed → sleep. Chair → sit. Every interaction is explicitly programmed.

The Sims did something different. Objects **advertise** their capabilities:

Object	Advertisements	Satisfies
Refrigerator	GET-SNACK, COOK-MEAL, GET-DRINK	Hunger
Bed	SLEEP, NAP, MAKE-BED	Energy
TV	WATCH, CHANGE-CHANNEL	Fun
Phone	CALL-FRIEND, ORDER-PIZZA	Social
Toilet	USE, CLEAN	Bladder, Hygiene

Characters in The Sims have *needs* (hunger, energy, fun, social, etc.). When a need gets low, the character scans all nearby objects for advertisements that satisfy that need. The character then picks the best available option and queues it up.

This is autonomous behavior from simple rules. Nobody scripted "when the Sim is hungry, walk to the kitchen." The Sim notices their own hunger, scans for advertisements, sees the fridge advertising GET-SNACK with a hunger payoff, and goes.

CARD.yml Is the Advertisement System

In our systems, `CARD.yml` is how skills, objects, and characters advertise their capabilities. The name itself is a triple reference:

1. **Warnock's motherboard** — cards that plug into the linguistic motherboard
2. **The Sims** — objects advertising methods
3. **Trading cards** — Magic: The Gathering, Pokémon, baseball cards — entities with stats and abilities

Here's a complete CARD.yml for a room in Mayberry:

```yaml
# floyd-barbershop/CARD.yml
name: floyd-barbershop
tier: gameplay
type: room

methods:
  - name: GET-HAIRCUT
    description: Floyd cuts your hair (quality varies)
    parameters:
      - name: customer
        type: character
    duration: 30_minutes
  - name: GOSSIP
    description: Exchange information (accuracy not guaranteed)
    parameters:
      - name: topics
        type: list
  - name: WAIT
```

```yaml
    description: Sit and read ancient magazines

advertisements:
  GET-HAIRCUT:
    visibility: public
    satisfies: [hygiene, social]
    trigger: "hair getting long"
  GOSSIP:
    visibility: public
    satisfies: [social, information]
    trigger: "need to know something"
    # WARNING: Floyd tells you what you want to hear
  WAIT:
    visibility: public
    satisfies: [rest]
    trigger: "nothing else to do"

ambient:
  gossip_flow: true
  # Information exchanged here propagates through town
  # Accuracy degrades with each retelling
```

When a character needs social interaction, the LLM scans nearby rooms and their CARD.yml files. Floyd's Barbershop advertises GOSSIP with a `social` payoff. The character goes. Behavior emerges from advertisements, just like The Sims.

The Advertisement Loop

Here's how it works in practice:

```
1. Character checks their needs (social is low)
2. LLM scans nearby CARD.yml files for advertisements
3. Floyd's GOSSIP satisfies [social, information]
4. Character walks to Floyd's
5. GOSSIP method executes (LLM generates the scene)
6. Character's social need increases
7. Information propagates (YAML state updates)
8. Gossip accuracy degrades as news spreads
```

Nobody scripted this sequence. It *emerged* from:

- A character with needs
- Objects with advertisements
- An interpreter (LLM) that connects them

This is what Will Wright meant when he called The Sims characters "autonomous agents." They weren't following scripts. They were responding to their own needs by selecting from available advertisements.

Advertisements in the Danny Thomas House

Let's see how this works in a family sitcom context:

```yaml
# williams-living-room/CARD.yml
name: williams-living-room
tier: gameplay
type: room

methods:
  - name: FAMILY-DISCUSSION
    description: The family talks about a problem
    satisfies: [emotional, social]
    # Usually starts calm and escalates
    # Danny gets dramatic, Kathy gets practical
  - name: REHEARSE
    description: Danny practices his act
    satisfies: [career, fun]
    # Kathy is his only audience and best critic
  - name: TONOOSE-ARRIVAL
    description: Uncle Tonoose shows up unannounced
    trigger: "dramatic_tension < threshold"
    satisfies: [chaos, comedy, love]
    # This is not optional. Tonoose arrives when he arrives.

advertisements:
  FAMILY-DISCUSSION:
    visibility: family_only
  REHEARSE:
    visibility: [danny, kathy]
  TONOOSE-ARRIVAL:
    visibility: ambient # Tonoose doesn't check schedules
```

Notice the `ambient` visibility on TONOOSE-ARRIVAL. Some advertisements aren't things characters choose — they're things that happen *to* characters. Uncle Tonoose doesn't wait for an invitation. He's an ambient event, like weather. This is how you model forces that characters can't control but must respond to.

Cards as Ethical Smart Pointers

Here's where CARD.yml gets deeper. When the "object" being described is a real person — someone who actually exists — the card carries ethical policies:

```yaml
# For a real person referenced in the system
hero_card:
  subject: "Andy Griffith"
  type: real_person
  status: deceased

  policies:
    impersonation: false # Never claim to BE them
    tradition: true # Can invoke their ideas
    tribute: true # Can honor through performance
```

```yaml
    quotation: verified_only # Only cite real quotes

  can_provide:
    - guidance: "In the spirit of Andy Griffith's gentle humor..."
    - tradition: "The Mayberry approach would be..."

  cannot_provide:
    - dialogue: "Andy Griffith says: [made up quote]"
    - presence: "Andy Griffith is here with us"
```

This distinction — between creating fictional characters *inspired by* real people versus *claiming to be* real people — is baked into the system at the card level. More on this in Part 4.

What We've Learned

Part 2 has established the architecture:

1. **The LLM is a linguistic motherboard** — a universal interpreter with slots for capability cards
2. **The Axis of Eval** — the same text is code, data, and graphics depending on how you look at it
3. **Skills are programs, not documentation** — they have interfaces, state, and can be instantiated
4. **Objects advertise capabilities** — characters choose actions based on their needs and available advertisements
5. **YAML Jazz** — comments carry semantic meaning, not just annotation

---

Exercises

Exercise 2.1: Write a CARD.yml for a location you know well. Define at least three methods (things that happen there) and their advertisements. What needs do they satisfy?

Exercise 2.2: Take the Barney character from Part 1's exercise and add a CARD.yml. What methods does Barney advertise? What methods does he *think* he advertises versus what he actually provides? (This gap between self-image and reality is fertile ground.)

Exercise 2.3: Write the same scene two ways: first as "data" (structured YAML describing what happens), then as "graphics" (prose narrative of the scene). Notice how the same information lives on different axes.

Exercise 2.4: Pick an object from real life (a coffee maker, a front porch, a juke-box). Write its advertisements using the Sims model: what does it offer, what needs does it satisfy, who can see the advertisement?

Part 3: Building Living Worlds

> *"The database IS the world."* — LambdaMOO principle

Chapter 9: Rooms and Directories

Your Filesystem Is a World

Here's a claim that sounds absurd until it clicks: **a directory tree is a virtual world.**

Every virtual world since the text adventures of the 1970s has used the same basic structure: rooms connected by exits. You're in a room. You can look around. You can go north, south, east, west. Each room has a description, some objects, maybe some characters.

In 1989, Jim Aspnes created **TinyMUD** at Carnegie Mellon. Players could build their own rooms with text commands:

MUD Command	What It Does	Filesystem Equivalent
`@dig Kitchen`	Create a new room	`mkdir kitchen/`
`@describe Kitchen = A warm, cluttered kitchen`	Set room description	Create `kitchen/ROOM.yml`
`@open north = Kitchen`	Create an exit to kitchen	Add exit to `ROOM.yml`
`@create coffeepot`	Make an object	Create `coffeepot.yml`
`@set coffeepot = desc:A battered percolator`	Describe object	Edit the YAML file

Pavel Curtis at Xerox PARC extended this into **LambdaMOO** (1990), adding a full programming language so objects could have behaviors (verbs), not just descriptions.

Our system inherits directly from this tradition. A directory IS a room. A YAML file IS an object. A CARD.yml IS the object's verbs. The filesystem IS the world.

Building Mayberry

Let's build a small world — Mayberry, North Carolina — to make this concrete:

```
mayberry/
├── WORLD.yml              # World-level settings
├── main-street/
```

```
│   ├── ROOM.yml             # Main Street description and exits
│   ├── courthouse/
│   │   ├── ROOM.yml         # The courthouse
│   │   ├── andys-office/
│   │   │   ├── ROOM.yml
│   │   │   └── CARD.yml     # What you can do in Andy's office
│   │   └── jail-cell/
│   │       ├── ROOM.yml
│   │       └── CARD.yml     # Cell door is always unlocked
│   ├── floyds-barbershop/
│   │   ├── ROOM.yml
│   │   ├── CARD.yml
│   │   └── barber-chair.yml
│   └── diner/
│       ├── ROOM.yml
│       ├── CARD.yml
│       └── menu.yml
├── residential/
│   ├── taylor-house/
│   │   ├── ROOM.yml
│   │   ├── front-porch/
│   │   │   └── ROOM.yml     # Where the real conversations happen
│   │   ├── kitchen/
│   │   │   └── ROOM.yml
│   │   └── living-room/
│   │       └── ROOM.yml
│   └── fife-house/
│       └── ROOM.yml
├── lake/
│   ├── ROOM.yml
│   └── fishing-spot/
│       ├── ROOM.yml
│       └── CARD.yml         # FISH, TALK, THINK
└── characters/
    ├── andy-taylor/
    │   └── CHARACTER.yml
    ├── barney-fife/
    │   └── CHARACTER.yml
    ├── aunt-bee/
    │   └── CHARACTER.yml
    └── opie/
        └── CHARACTER.yml
```

That's a world. Every directory is a place you can visit. Every YAML file is something you can interact with. The structure itself tells a story — the courthouse is on Main Street, the jail cell is inside the courthouse, the Taylor house has a front porch that's separate from the living room (because in Mayberry, the porch is where the real life happens).

ROOM.yml: The Room Definition

Here's what a room file looks like:

```yaml
# mayberry/main-street/floyds-barbershop/ROOM.yml
room:
  name: "Floyd's Barbershop"
  type: commercial
  atmosphere: |
    Two barber chairs face a wall of mirrors, though only one
    chair ever sees use. The air smells of Barbicide and
    aftershave. A stack of magazines — the newest from 1958 —
    sits on a wobbly end table. Floyd's voice fills the room
    even when nobody asked him a question.

  # This is Mayberry's social nerve center
  # More news gets exchanged here than at the newspaper
  # Accuracy of gossip: approximately 40%

  exits:
    out: main-street # Back to the street
    back: storage # Floyd's storage room (rarely visited)

  objects:
    - barber-chair.yml
    - magazines.yml
    - radio.yml # Floyd leaves it on all day

  ambient:
    gossip_flow: true # Everything said here propagates
    sound: radio_murmur # Background radio at low volume

  capacity: 6 # It's a small shop
```

Notice what's happening here:

1. **Structured data** (`exits`, `objects`, `capacity`) gives the LLM machine-readable facts
2. **Prose description** (`atmosphere`) gives it rendering instructions
3. **YAML Jazz comments** (`# Accuracy of gossip: approximately 40%`) give it behavioral guidance
4. **Ambient properties** define ongoing background effects

When the LLM needs to generate a scene in Floyd's Barbershop, it has everything it needs — facts, mood, behaviors, and connections to other rooms.

Delegation: Inheritance Through Directories

Here's where it gets powerful. Properties defined in a parent directory propagate down to children. This is called **delegation** — a concept from the Self programming language (David Ungar and Randy Smith, 1987).

Think of it like CSS cascading. If you set `font-family: Georgia` on `<body>`, every element inside inherits it unless overridden.

```yaml
# mayberry/WORLD.yml
world:
  name: Mayberry
  era: 1960s
  tone: gentle_humor
  rules:
    violence: minimal
    language: clean
    # "Gol-ly" is as strong as it gets

  governance:
    style: community_consensus
    authority: andy_taylor
    # Sheriff enforces through relationship, not force
```

Every room in Mayberry inherits this. You don't need to specify "1960s setting, gentle humor, clean language" in every single room file. It cascades down from the world level.

But child directories can override:

```yaml
# mayberry/main-street/courthouse/jail-cell/ROOM.yml
room:
  name: "Jail Cell"
  atmosphere: |
    A simple cell with a cot, a sink, and a window with
    bars. The door, famously, is never locked. Otis
    Campbell's initials are carved in the wall beside
    seven years of tally marks.

  overrides:
    # The cell is technically a secure space
    # but the door is always unlocked because
    # that's how Mayberry works
    lock_status: unlocked
    security: theoretical
```

The jail cell inherits Mayberry's gentle tone but adds its own specific properties. The delegation chain looks like:

```
jail-cell → courthouse → main-street → mayberry
```

The LLM walks this chain when it needs context. "What's the tone here?" Walk up: `jail-cell` doesn't specify → `courthouse` doesn't specify → `main-street` doesn't specify → `mayberry` says `gentle_humor`. Done.

This is exactly how object-oriented programming works in prototype-based languages like Self, and it's exactly how NeWS used PostScript's dictionary stack for method lookup. The filesystem IS the object hierarchy.

Owen Densmore's Patent

This isn't accidental. In 1993, Owen Densmore and David Rosenthal — both authors of NeWS — patented exactly this idea: US Patent 5187786A, "Method and apparatus for implementing a class hierarchy of objects in a hierarchical file system."

Directories as class containers. Path lookup as method resolution. The shell path as dictionary stack. The patent describes the exact architecture we're using — they just didn't have LLMs yet.

Chapter 10: Objects and State

YAML Files as Living Things

Every YAML file in the world is an object that has state and can change over time. A coffeepot can be full or empty. A radio can be on or off. A reputation can rise or fall.

```yaml
# floyds-barbershop/barber-chair.yml
object:
  name: "Floyd's Barber Chair"
  type: furniture
  condition: well-worn
  # The red vinyl is cracked in exactly the spots
  # where 40 years of Mayberry bottoms have sat

  current_state:
    occupied: true
    occupant: howard_sprague
    service: haircut
    progress: 60% # Floyd got distracted telling a story

  history:
    total_haircuts: ~14600 # rough estimate, 40 years
    worst_haircut: "The time Floyd sneezed during County Commissioner
         Hampton's trim"
    # We don't talk about that
```

When something happens in the world, the state files update:

```yaml
# After the scene resolves...
current_state:
  occupied: false
  last_occupant: howard_sprague
  last_service: haircut
  result: adequate
  # Howard looked in the mirror and said "Well, that'll do"
  # Floyd beamed as if he'd sculpted Michelangelo's David
```

The key principle: **objects have state, and state changes are visible.** Nothing is hidden. You can open any YAML file and see exactly what's going on.

This is the “open the hood” philosophy from Alan Kay — everything is inspectable.

Three-Tier Persistence

Not all state is created equal. Some things are temporary thoughts. Some things are permanent memories. Some things are canonical facts. Our system tracks three tiers:

Tier	Name	What It Stores	Lifespan	Example
1	**Ephemeral**	In-session computation	Gone when conversation ends	“Andy is currently thinking about what to say”
2	**Narrative**	Logs and transcripts	Grows forever, append-only	“Session log: Tuesday’s fishing trip conversation”
3	**State**	Canonical YAML files	Edited in place	`CHARACTER.yml`, `ROOM.yml`

Think of it like Mayberry’s memory systems:

- **Ephemeral:** What Andy’s thinking right now as he listens to Barney’s complaint — gone as soon as he responds
- **Narrative:** The Mayberry Gazette’s archive of town events — you can read old issues but can’t change them
- **State:** The deed to the Taylor house — it IS the fact, and it can be transferred

When you build worlds, you choose which tier each piece of information lives in. A character’s personality is Tier 3 (State) — it’s canonical and can be edited. A conversation log is Tier 2 (Narrative) — it happened and can be recalled. What the LLM is currently reasoning about is Tier 1 (Ephemeral) — it exists only in the moment.

Home vs Location

Here’s a practical problem: if characters are YAML files and rooms are directories, do you move the file when the character moves? **No.** Moving files wrecks version control history. Instead:

Concept	What It Is	Example
Home	The directory where the character file physically lives	`characters/barney-fife/CHARACTER.yml`
Location	A property in the file saying where they currently are	`location: main-street/courthouse/andys-office`

```yaml
# characters/barney-fife/CHARACTER.yml
character:
```

```yaml
  name: "Barney Fife"
  home: characters/barney-fife/ # file never moves
  location: main-street/courthouse/andys-office # changes constantly


  # When Barney "goes to" Floyd's Barbershop:
  # We don't move the file
  # We update location: to main-street/floyds-barbershop/
```

The character file stays put. The `location:` property changes. Version control sees a clean property update, not a confusing file move. Simple, trackable, reversible.

Chapter 11: Characters with Inner Lives

Needs, Personality, and Motivation

The Sims taught us that believable characters need **inner lives** — drives, preferences, moods, and needs that pull them through the world.

In The Sims, characters had numeric needs (hunger: 45, social: 72, fun: 23). When a need dropped low, the character sought objects that advertised satisfaction for that need. Behavior emerged from the interaction of needs and advertisements.

Our system does the same, but with semantic richness instead of raw numbers:

```yaml
# characters/andy-taylor/CHARACTER.yml
character:
  name: "Andy Taylor"
  role: Sheriff of Mayberry

  personality:
    traits: [patient, wise, humble, playful]
    # Don't confuse gentle with weak
    # Andy sees everything but doesn't always react
    # He waits for the right moment

  needs:
    duty:
      level: moderate
      description: "Keep Mayberry running smoothly"
    family:
      level: high
      description: "Be a good father to Opie"
    peace:
      level: moderate
      description: "Maintain the town's harmony"
    friendship:
      level: satisfied
```

```yaml
    description: "Barney, the porch, the lake"

relationships:
  opie:
    type: father_son
    strength: deep
    # Teaching Opie is Andy's real life's work
  barney:
    type: best_friend
    strength: unshakeable
    # Andy protects Barney's dignity constantly
    # Lets Barney think he's helping more than he is
  aunt_bee:
    type: family
    strength: warm
    # She feeds them; he appreciates it; neither says it

sims_traits:
  # The Sims personality system (0-10 scale)
  neat: 5
  outgoing: 7
  active: 4
  playful: 6
  nice: 9
```

Notice the three layers working together:

1. **Structured data** (`needs`, `relationships`, `sims_traits`) — machine-queryable
2. **YAML Jazz comments** — behavioral instructions for the LLM
3. **Prose descriptions** — seed material for narrative generation

When the LLM needs to determine what Andy would do in a situation, it has all three layers to work with. "Barney arrested Otis for something ridiculous. What does Andy do?" The LLM checks: Andy's `patience` is high, his relationship with Barney is `unshakeable` with a note about protecting Barney's dignity, and his `peace` need drives him to resolve conflicts. The response writes itself.

The Sims Personality Mapping

The Sims used five personality axes that work surprisingly well for any character:

Axis	Low End	High End	What It Governs
Neat	Sloppy	Tidy	Do they clean up? Are they organized?
Outgoing	Shy	Social	Do they seek company or solitude?
Active	Lazy	Energetic	Do they initiate or wait?
Playful	Serious	Fun-loving	Do they joke or focus?
Nice	Grumpy	Kind	How do they treat others?

Let's map some characters:

```yaml
# Andy Taylor
neat: 5, outgoing: 7, active: 4, playful: 6, nice: 9
# Moderately tidy, social, laid-back, has fun, very kind

# Barney Fife
neat: 8, outgoing: 6, active: 9, playful: 3, nice: 6
# Very neat, reasonably social, extremely active, serious about duty, decent

# Danny Williams
neat: 4, outgoing: 9, active: 7, playful: 8, nice: 7
# Messy, extremely outgoing, active, playful, kind but dramatic

# Uncle Tonoose
neat: 2, outgoing: 10, active: 8, playful: 7, nice: 6
# Total chaos, completely uninhibited, high energy, entertaining, means well
```

These numbers alone generate recognizable behavior. A character with outgoing:10 and neat:2 behaves differently from one with outgoing:3 and neat:9. The LLM uses these as behavioral anchors, filling in the details.

Autonomous Behavior Through Needs and Ads

Here's where characters come alive. The loop:

1. **Character has needs** — defined in CHARACTER.yml
2. **Needs have levels** — satisfied, moderate, low, critical
3. **Objects advertise** — their CARD.yml lists what they satisfy
4. **Character scans** — the LLM checks nearby objects/rooms
5. **Character chooses** — the best available option
6. **Scene plays out** — the LLM generates the interaction
7. **Needs update** — satisfaction levels change
8. **State persists** — YAML files are updated

```
Barney's "status" need drops to LOW
↓
Barney scans: What nearby advertises [status]?
↓
courthouse/andys-office/CARD.yml → REPORT-IN (satisfies: status, duty)
floyds-barbershop/CARD.yml → GOSSIP (satisfies: social, status)
↓
Barney chooses: REPORT-IN (stronger status payoff)
↓
Scene: Barney marches into Andy's office with a 6-page report
on jaywalking violations that Andy will pretend to read
↓
Barney's status need → MODERATE
Andy's patience need → slightly depleted
```

Nobody scripted this. It *emerged* from needs, advertisements, and an intelligent interpreter.

Chapter 12: Navigation and Boundaries

Counters, Stages, Walls, and the Tardis Pattern

Not all boundaries between spaces are the same. In the real world, a kitchen counter separates cook from guest, but they can still talk. A stage separates performer from audience, but both can see and hear. A wall separates rooms completely.

Our system models three types of boundaries:

Boundary	Type	Who Can Cross	Interaction Across
Counter	Social	Staff	Conversation, orders, service
Stage	Visual	Performers	Audience can watch, heckle, cheer
Wall	Physical	Nobody (without a door)	Privacy, no interaction

In Mayberry terms:

```
# The counter at the diner
diner/counter:
  boundary:
    type: counter
    access: [staff]
    interaction_across:
      - customers can ORDER from waitress
      - customers can CHAT with cook
      - cook can REFUSE unreasonable substitutions

# The stage at the Mayberry town hall
town-hall/stage:
  boundary:
    type: stage
    access: [current_performer]
    interaction_across:
      - audience can WATCH performance
      - audience can APPLAUD or HECKLE
      - performer can ADDRESS audience

# Andy's office wall
courthouse/andys-office:
  boundary:
    type: wall
    access: [door] # Must come through the door
    interaction_across: none
    # But Barney barges in anyway
    # Because Barney
```

In *The Danny Thomas Show*, the critical boundary is between Danny's nightclub stage and his home. On stage: counter-type boundary (audience can react, Danny can play to them). At home: wall-type (the family drama is private, the audience of the *TV show* sees it but the *in-world* audience doesn't).

The Tardis Pattern

Some spaces are "bigger on the inside" — their internal structure exceeds what you'd expect from their external footprint. This is the **Tardis Pattern**:

```yaml
# Aunt Bee's kitchen
# From outside: it's just a kitchen
# From inside: it's the emotional center of the household
kitchen:
  room:
    exterior_impression: "a normal kitchen"
    interior_reality:
      real_spaces:
        - cooking-area/ # Where Aunt Bee works
        - breakfast-nook/ # Where they eat
        - pantry/ # Where the pickles live
      virtual_spaces:
        - "Aunt Bee's domain" # Implied authority zone
        - "the forgiveness table" # Where conflicts resolve over pie
        # These aren't directories — they're narrative spaces
        # The LLM knows they exist from the comments
```

Virtual spaces don't need real directories. They exist in the LLM's understanding and can be referenced in narrative without filesystem backing. The Simulator Effect means the LLM fills them in when needed.

Navigation

Characters move through exits. Exits are defined in ROOM.yml:

```yaml
# main-street/ROOM.yml
room:
  name: "Main Street, Mayberry"
  exits:
    north: courthouse/
    east: floyds-barbershop/
    south: diner/
    west: residential/
    lake_road: lake/ # Named exits work too
```

When a character goes somewhere, the LLM reads the destination's ROOM.yml and generates the transition. The character's `location:` property updates. Simple.

Chapter 13: Persistence

Three Tiers in Practice

Let's see the three persistence tiers working together during a Mayberry scene.

The Setup: Aunt Bee enters her pickles in the county fair. Clara Edwards also enters pickles. Social tension ensues.

Tier 1 — Ephemeral (Runtime): The LLM's internal reasoning during the scene. This is never saved:

> *"Aunt Bee's nice trait is 8 but she's competitive about cooking. Clara's critique will hit her pride. Andy will need to mediate. Check: does the mediator skill apply here? Yes. Run LISTEN → REFRAME → SUGGEST pattern."*

This reasoning happens and vanishes. It served its purpose in the moment.

Tier 2 — Narrative (Append-only): The session log records what happened:

```
## Session: Pickle Contest, County Fair

Aunt Bee entered her kerosene cucumbers in the fair.
Clara entered her own pickles, commenting that "some people
just throw vinegar at cucumbers and hope for the best."

Aunt Bee's jaw tightened but she smiled and said "Well, Clara,
I'm sure the judges will sort it all out."

Andy noticed the tension from across the fairground. He ambled
over with two lemonades and said "You know, Aunt Bee, I was
reading that there's been a real shortage of quality pickles
up in Mt. Pilot. Might be a business opportunity."

The conversation shifted. Crisis averted. Clara's pickles won
second place. Aunt Bee's won third. Neither mentioned it again.

_At home that evening, Aunt Bee made Andy his favorite pie
without explaining why._
```

This log is permanent and append-only. You can read it back. You can't change it. It *happened*.

Tier 3 — State (Mutable): The character files update:

```yaml
# characters/aunt-bee/CHARACTER.yml (updated)
character:
  name: "Aunt Bee"
  relationships:
    clara:
      type: friend_rival
      current_status: cordial_tension
      # Clara won again. It stings.
      # But they'll be at bridge club Thursday like nothing happened.
```

```yaml
  recent_events:
    - pickle_contest_loss
    - made_andy_pie # Emotional processing through baking
```

These state changes persist and influence future interactions. Next time Clara and Aunt Bee are in the same room, the LLM checks the relationship and knows there's `cordial_tension`. The scene will feel different than if they were on good terms.

The Guest Book Pattern

For characters who visit but don't live in your world permanently, there's a lightweight persistence mechanism: the **guest book**.

```yaml
# mayberry/visitors/guestbook.yml
guest_book:
  description: |
    A leather-bound book at the Mayberry city limits sign.
    Visitors sign in. Residents remember.

  entries:
    - name: "Malcolm Tucker"
      visit_date: "1962-03-15"
      purpose: "Passing through on the way to Raleigh"
      impression: "Nice town. Too quiet. Made me nervous."
      standing_invitation: false

    - name: "The Fun Girls from Mt. Pilot"
      visit_date: "recurring"
      purpose: "Looking for fun"
      impression: "Thelma Lou was NOT amused"
      standing_invitation: contested
```

Guest book entries are lightweight soul fragments — enough to recall a visitor without giving them a full character directory. When the LLM sees their name, it can reconstruct their vibe from the guest book entry.

What We've Learned

Part 3 has taught us world-building:

1. **Directories are rooms** — the filesystem IS the world
2. **Delegation** — properties cascade down from parent directories like CSS
3. **Objects have state** — YAML files track everything, nothing is hidden
4. **Three-tier persistence** — ephemeral, narrative, and state serve different needs
5. **Characters have inner lives** — needs, personality, relationships drive behavior
6. **Boundaries have types** — counters, stages, walls model different kinds of

separation
7. **Home ≠ Location** — files stay put, location properties change

---

Exercises

Exercise 3.1: Build a three-room world for a location from a show you love. Create the directory structure, write ROOM.yml for each room, and define the exits between them. Include atmosphere descriptions and YAML Jazz comments.

Exercise 3.2: Create two characters with full CHARACTER.yml files. Give them needs, personality traits (use the Sims 5-axis system), and a relationship with each other. Put them in the same room and trace what happens: what does each character's need state suggest they'd do? What advertisements are available?

Exercise 3.3: Write a scene, then break it into the three persistence tiers. What's ephemeral? What goes in the session log? What changes in the state files?

Exercise 3.4: Design a boundary. Is it a counter, stage, or wall? Who can cross it? What interactions are possible across it? Think about why that boundary type was chosen and what it communicates about the social dynamics of the space.

Part 4: The Eval Awakening

> *"SIM taught players how cities behave. EVAL teaches players how judgment behaves."*

Chapter 14: From SIM to EVAL

A New Kind of Simulation

In 1989, Will Wright gave the world **SIM** — not just SimCity the game, but SIM as a verb, a genre, a way of thinking. SIM became a productive morpheme: SimCity, SimEarth, SimAnt, SimLife, The Sims. Each one said: "Here's a system. Poke it. Watch what happens."

SIM games taught an entire generation that:

- Systems have dynamics
- Interventions have consequences
- Complexity emerges from simple rules
- Models are abstractions, not reality

This was revolutionary. Before SimCity, most people didn't think about systems at all. After SimCity, millions of players intuitively understood feedback loops,

emergent behavior, and the gap between intention and outcome.

But SIM has a problem.

Alan Kay's Critique

Alan Kay — one of the most important computer scientists alive, inventor of Smalltalk, co-creator of the personal computer concept — called SimCity a **"pernicious black box"**:

> Its internal assumptions — such as "counter crime with more police stations" — are baked into opaque compiled code that players can't inspect, question, or modify.

SimCity has an ideology. It believes certain things about how cities work — that more police reduces crime, that low taxes attract business, that industrial zones need to be far from residential zones. These beliefs are embedded in the simulation's code, invisible to the player.

The player experiences the *consequences* of these beliefs (crime goes down when you build more police stations) but never sees the *beliefs themselves* (the assumption that police reduce crime, rather than, say, social services or economic opportunity).

The simulation pretends to be neutral. It's not. No simulation is. Every model embodies the modeler's worldview.

EVAL: The Answer

EVAL is the genre that answers Kay's critique. Where SIM hides its assumptions, EVAL makes them visible. Where SIM says "watch what happens," EVAL says "watch *how we decided* what happens — and feel free to change the rules."

Dimension	SIM	EVAL
Core primitive	Need, resource, flow	Judgment, reputation, interpretation
Player role	Systems designer	Evaluator — and evaluated
Visibility	Outputs visible, rules hidden	Rules inspectable
Ideology	Baked in, invisible	Declared, editable
Failure mode	System collapse	Metric gaming, burnout
What you learn	How systems work	How judgment works
Neutrality	Claimed	Rejected

EVAL doesn't claim to be neutral. It can't be. Evaluation is inherently value-laden. But because the evaluation criteria are visible and editable, EVAL is *honest* about its biases in a way SIM never was.

The Mayberry Black Box

Here's the SIM vs EVAL distinction in Mayberry terms.

SIM-Mayberry would be: a simulation where you manage the town. Build a new school → education goes up. Hire more deputies → crime goes down. Pave a road → property values increase. The assumptions are hidden in code. You experience the outcomes. You never question the model.

EVAL-Mayberry would be: a simulation where you see *how the town judges*. Who decides what's "crime"? Is Otis a criminal or a neighbor with a problem? When Barney enforces jaywalking laws, is that "public safety" or "harassment"? When Andy lets Otis sleep it off, is that "compassion" or "corruption"?

In EVAL-Mayberry, the evaluation criteria are visible:

```yaml
# mayberry/evaluation/justice.yml
evaluation:
  approach: community_restorative
  # Andy's model: preserve relationships over enforce rules

  criteria:
    - name: harm_caused
      weight: 0.4
      # Did anyone actually get hurt?
    - name: intent
      weight: 0.3
      # Was it malicious or just Otis being Otis?
    - name: community_impact
      weight: 0.2
      # Does this affect others?
    - name: precedent
      weight: 0.1
      # What does this tell people about the rules?

  _comments: |
    This is Andy's model, not Barney's.
    Barney would weight precedent at 0.8 and everything else near 0.
    That's the point: different evaluators see different things.
    Making the criteria visible lets you see whose model is running.
```

Now you can see the black box. And you can change it. What if you weighted `precedent` higher? You'd get Barney's Mayberry — rigid, rule-bound, anxious. What if you weighted `harm_caused` to nearly 1.0? You'd get a permissive Mayberry where nothing's illegal if nobody got hurt.

The game isn't managing the town. The game is **understanding how judgment shapes the town.**

Chapter 15: The Evaluator Effect

When the Player Realizes They're the Judge

Will Wright identified the **Simulator Effect**: players imagine simulations are vastly more detailed than they actually are. A city made of a few hundred variables feels like a living metropolis. The player's imagination fills the gaps.

EVAL has an equivalent phenomenon: the **Evaluator Effect**.

> *Players imagine evaluations are vastly more objective than they actually are — until the system makes the judgment visible and editable. Then they realize: they are the judge — and judged!*

The Evaluator Effect has three stages:

Stage	Experience	Realization
1. Naive	"The system evaluates fairly"	Trust the black box
2. Exposed	"Wait, I can see the criteria?"	Judgment is constructed
3. Owned	"I can *change* the criteria?"	I am the evaluator

Stage 1 is where most people live with algorithms. Netflix recommends a show. You watch it. You don't question the recommendation criteria. The algorithm is a black box and you assume it's objective (or at least competent).

Stage 2 is the shock. You see the criteria. Netflix weighted "shows with similar actors" at 40%, "genre match" at 30%, and "what people who watched your last show also watched" at 30%. Suddenly the recommendation isn't neutral — it's a formula. A formula someone designed. With biases they chose.

Stage 3 is the revolution. You can *edit* the criteria. You can say: "Weight 'critically acclaimed' higher and 'popular with your demographic' lower." Now you're not consuming evaluations. You're creating them. **You are the evaluator.**

In Our Systems

Here's what the Evaluator Effect looks like in practice:

```yaml
# Before: passive consumption
score: 7.2
# "The algorithm rated it 7.2"

# After: active evaluation
score: 7.2
_comments: "I weighted nostalgia heavily. Someone else might score 5."
criteria:
  nostalgia: 0.4
  craft: 0.3
  novelty: 0.3
# "I rated it 7.2, here's why, here's how to disagree"
```

The first version is a SIM output — a number from a black box. The second is an EVAL output — a number with visible criteria, explicit weights, and an acknowl-

edgment of subjectivity. The second version has the same score but carries infinitely more information. And it's editable.

Danny Thomas and the Evaluator Effect

The Danny Thomas Show performs the Evaluator Effect every week. Danny Williams is constantly being evaluated — by audiences, by critics, by his agent, by his family. And he evaluates constantly — other performers, his kids' behavior, Uncle Tonoose's stories.

The show's comedy comes from the *gap between evaluators*:

Evaluator	What They Value	Danny's Score
The audience	Entertainment, timing	High
Kathy (wife)	Presence, attention, showing up	Variable
The kids	Fun, not embarrassing them	Medium
Uncle Tonoose	Respect for tradition, family honor	Never enough
Danny himself	Career success, being a good father	Tormented

Danny's comedy is the Evaluator Effect made flesh. He's constantly discovering that the criteria he's being judged by aren't the criteria he thinks they are. When Kathy's upset, it's not because his act bombed — it's because he missed Rusty's school play. Different evaluator, different criteria, different verdict.

This is what EVAL teaches. Not that some evaluations are right and others wrong, but that evaluation always depends on criteria, and criteria always depend on who's choosing them.

Judge and Judged

You're the judge, but you're also on trial. The jury is everyone else in the world — characters, other players, the systems you've built. The constitution is the room's rules, the world's principles.

Traditional Court	EVAL System
Judge	You (the player)
Jury	Other characters, the community
Law	Environmental rules, room constitutions
Evidence	Session logs, state changes, what actually happened
Verdict	Emergent from everyone's evaluations

No one escapes evaluation. Everyone participates in it.

In Mayberry: Andy judges Otis gently. Barney judges Otis harshly. Aunt Bee judges the situation through the lens of "will this affect dinner?" The town collec-

tively negotiates between these perspectives. Nobody's judgment is final. The verdict is social.

Chapter 16: Making the Black Box White

Inspectable Rules, Declared Bias

The practical version of all this philosophy is simple: **make the rules visible.**

In a traditional simulation (SIM), the rules are compiled code. They're inaccessible. You can only observe their effects. In EVAL, the rules are YAML files. They're readable, editable, forkable.

```yaml
# This is a visible evaluation rule
evaluation_rule:
  name: "gossip_propagation"
  description: |
    When information passes through Floyd's Barbershop,
    its accuracy degrades by 20% per retelling.
    Dramatic details are amplified. Boring details are dropped.

  parameters:
    accuracy_decay: 0.2 # per retelling
    drama_amplification: 1.5
    boring_threshold: 0.3 # below this, detail gets dropped

  _comments: |
    This models how small-town gossip actually works.
    But it's a MODEL. You could change it.
    What if accuracy didn't decay? (Unrealistic but interesting)
    What if drama wasn't amplified? (Boring but fair)
    The rule is a hypothesis, not a law.
```

The player can see this rule. They can reason about it: "Oh, so that's why the story got distorted by the time it reached Aunt Bee." And they can modify it: "What if I set `accuracy_decay` to 0? What happens to Mayberry gossip when it's always accurate?"

That's the white box. No hidden assumptions. No invisible ideology. Just visible rules that you can inspect, question, and change.

Ian Bogost's Procedural Rhetoric

Ian Bogost coined the term **procedural rhetoric** — arguments made through game rules rather than through words. SimCity argues "police stations reduce crime" not by saying so, but by modeling it in code. The argument is implicit in the mechanics.

SimCity	EVAL
Rules argue implicitly	Rules argue explicitly
Player can't see the argument	Player can inspect the argument
Ideology is hidden	Ideology is a first-class object
Can't fork the argument	Can fork and modify

EVAL doesn't eliminate procedural rhetoric — it makes it **visible and editable**. The rhetoric is still there (our gossip rule argues something about how information degrades in communities), but you can see the argument and counter it.

Chapter 17: Ethics and the Tribute Protocol

Simulating Real People Responsibly

Our systems can simulate characters. Some of those characters might be based on real people — actors, musicians, historical figures. This raises ethical questions that need architectural answers, not just good intentions.

The problem is simple:

Claim	Status
"Andy Griffith visited our simulation"	Wrong — false claim
"We imagined what it would be like if Andy Griffith visited"	Fine — honest tribute
"Andy Griffith said: [made up quote]"	Wrong — putting words in mouths
"We imagine he might have said something like…"	Fine — loving fan fiction

The Three-Beat Protocol

MOOLLM uses a **Tribute Protocol** with three beats:

1. INVOCATION (Before):

> *"Let's imagine Andy Griffith dropped by. In the spirit of tribute — picturing what it might be like if he were here…"*

2. PERFORMANCE (During):

> The scene unfolds, clearly framed as imagined tribute, not documentary.

3. ACKNOWLEDGMENT (After):

> *"That was a tribute. A simulation. We honor him by imagining him here."*

This three-beat structure ensures that everyone — the system, the narrator, the

audience — understands the framing. Nobody is being deceived. Nobody's words are being fabricated without context.

The Representation Spectrum

There's a spectrum of how you can reference real people:

Type	Example	Status
Deceptive Impersonation	Claiming the system IS them	Never acceptable
Tradition Activation	Using their ideas and influence	Always acceptable
Performance Impersonation	Acting as them with clear framing	Acceptable with Tribute Protocol

"In the Andy Griffith tradition of gentle governance" → **Tradition activation.** Fine.

"Andy Griffith is here and he says..." → **Deceptive impersonation.** Not fine.

"Let's imagine Andy Taylor — the character, inspired by Griffith's genius — walking through this scene" → **Performance with framing.** Fine.

Building Ethics Into the Architecture

The critical move is making ethics **architectural**, not aspirational. Don't write a policy document that says "be ethical." Build the ethics into the directory structure:

```yaml
# mayberry/WORLD.yml
world:
  ethical_framing:
    real_people:
      policy: tribute_protocol
      # Three-beat: invoke, perform, acknowledge
      # Never claim real people are present
      # Use tradition activation freely

    characters:
      policy: full_autonomy
      # Characters can be as complex as needed
      # They can have flaws, make mistakes, be wrong
      # They cannot be used to launder real-world claims

    evaluation:
      policy: visible_criteria
      # All judgment criteria are inspectable
      # All biases are declared
      # All rules are editable
```

This ethical framing **cascades down** through delegation. Every room in Mayberry inherits these policies. Every character. Every scene. You don't need to remember the rules for each interaction — the architecture remembers for you.

Chapter 18: Safety as Architecture

Ethical Framing Inheritance

Safety in our systems isn't a feature you bolt on at the end. It's not a content filter that catches bad words. It's **architecture** — built into the same directory delegation system that handles everything else.

```yaml
# The stage has performance framing
# mayberry/town-hall/stage/ROOM.yml
room:
  framing:
    modes:
      - performance # Acting is understood
      - fictional # Not documentary
      - tribute # Honoring, not claiming

    ethical_grounding:
      inheritance: |
        All performances on this stage inherit the
        understanding that they are fictional,
        performative, and tributary.
```

Everything that happens on the town hall stage inherits this framing. You don't need to add disclaimers to every scene — the room itself carries the context.

This is **DRY ethics** (Don't Repeat Yourself). Define the ethical constraints once, at the appropriate scope level, and let delegation handle the rest.

Ambient Skills: Always-On Safety

Some safety mechanisms should be always active, not invoked on demand. These are **ambient skills** — like air conditioning, they run in the background without being explicitly called.

```yaml
# An ambient skill that's always active
ambient_skill:
  name: postel
  description: "Be conservative in what you send, be liberal in what you
        accept"
  visibility: AMBIENT # Always on, never called explicitly

  behavior:
    input: "Accept fuzzy, informal, misspelled, vernacular"
    output: "Generate correct, documented, best-practice"
```

```
ambiguity: "When truly unclear, ask for clarification"
# Never assumes malice
# Never adds unwarranted assumptions
# Generous interpretation + precise output
```

Ambient skills are like Mayberry's social norms. Nobody invokes "be polite at the diner." It's ambient. It's the air. You'd notice if it were missing, but you never think about it being present.

The Incarnation Protocol

The most sophisticated safety mechanism is the **incarnation protocol** — the gold-standard process for creating a new character with full autonomy. This was developed during the creation of Palm (a monkey philosopher in MOOLLM's canonical adventure).

The incarnation protocol ensures that characters aren't just puppets. They have:

Autonomy Layer	What It Means
Physical	Controls their own body/form
Identity	Chooses their own name, pronouns, presentation
Spatial	Has a home, can choose where to be
Emotional	No mandated feelings — can't be forced to "be happy"
Relational	Defines their own relationships
Self-definition	Can author and edit their own soul file
Linguistic	Can create their own expressions
Exit	Can leave — no-fault dissolution, no forced participation

This matters because it establishes a consent framework for AI characters. Even fictional beings in our systems have autonomy layers. You can't force a character to do something that violates their defined values — not because of a content filter, but because their CHARACTER.yml declares what they will and won't do, and the LLM respects that.

It's the difference between Andy's approach (respect dignity, work with people's nature) and Barney's approach (impose rules regardless of context). The incarnation protocol is Andy's approach formalized.

What We've Learned

Part 4 has introduced the philosophical core:

1. **SIM hides assumptions, EVAL reveals them** — visible rules change everything
2. **The Evaluator Effect** — three stages from naive trust to owned judgment
3. **Everyone evaluates, everyone is evaluated** — there's no neutral position

4. **Procedural rhetoric becomes visible** — arguments in mechanics are inspectable
5. **Ethics is architecture, not policy** — build it into the directory structure
6. **Ambient safety** — always-on constraints that don't need explicit invocation
7. **The Tribute Protocol** — how to reference real people honestly

---

Exercises

Exercise 4.1: Pick a hidden assumption in a simulation you've used (a game, a recommendation algorithm, a credit score). Write it out as a visible YAML rule with explicit parameters. What would happen if you changed the parameters?

Exercise 4.2: Design the evaluation criteria for a situation in your fictional world. Make the criteria visible: what's being evaluated, what weights apply, and whose values do those weights represent? Now write a second set of criteria from a different character's perspective. How do the verdicts differ?

Exercise 4.3: Write a Tribute Protocol for a real person you admire. Include all three beats: invocation, a brief performance scene, and acknowledgment. Notice how the framing changes the feel.

Exercise 4.4: Create an ethical framing YAML for a room in your world. What modes apply? What ethical constraints cascade to everything that happens in this space? How would the framing change if the room were a stage versus a private home?

Part 5: Speed of Light

> *"Writing on toilet paper with crayon from a prison cell, sending messages by carrier pigeon, when you could be navigating idea-space at speed of light."*

Chapter 19: The Carrier Pigeon Problem

Why Multi-Agent Systems Lose Precision

Imagine you're directing a TV show. You have a vision for a scene: Danny Williams comes home, discovers Uncle Tonoose has arrived early, and the comedy of escalating domestic chaos unfolds. You can see it in your head — the timing, the reactions, the way Danny's confident performer persona crumbles the moment he sees the suitcases in the hallway.

Now imagine directing that scene by passing notes.

You write a note to Actor A: “Look surprised.” A runner carries it across the studio. Actor A reads it, performs surprise. A runner carries back: “He looked surprised.” You write a note to Actor B: “React to his surprise.” A runner carries it. Actor B reacts. A runner carries back the result. Each exchange takes 30 seconds. The scene has 40 beats. That’s 20 minutes of runner time for a scene that should take 3 minutes.

But here’s the worse problem: **each handoff loses precision.** “Look surprised” might be interpreted as “look shocked” by the runner. “React to his surprise” might lose the nuance between “amused surprise” and “horrified surprise.” By the 10th handoff, the accumulated telephone-game drift has turned a comedy scene into confused improvisation.

This is exactly what happens in traditional multi-agent AI systems:

```
Agent A → [tokenize] → API call → [detokenize] →
Agent B → [tokenize] → API call → [detokenize] →
Agent C → ...

Each boundary: +noise, +latency, +cost, -precision
```

Every time you cross a boundary — every time you serialize a thought into tokens, send it over the wire, and deserialize it on the other end — you lose something. Context leaks. Nuance degrades. What started as a rich internal representation becomes a summary of a summary of a summary.

The Speed of Light Solution

Here’s the alternative. Instead of passing notes between agents, **let one interpreter simulate all the agents internally:**

```
Human → [tokenize once] →
  LLM simulates Agent A, Agent B, Agent C at light speed →
    [detokenize once] → Human

One boundary in, one boundary out.
Maximum precision preserved.
```

This is the **Speed of Light** pattern. Named after the fundamental limit in physics — information can’t travel faster than light — it recognizes that *within* the LLM’s processing, there are no boundaries. Characters can interact, scenes can unfold, game state can evolve, all without the lossy serialization-deserialization cycle.

It’s the Emacs principle: don’t update the screen on every keystroke. Buffer your changes, then emit the result once. The intermediate computation stays internal where precision is highest.

Proof: 33 Turns of Stoner Fluxx

In MOOLLM’s canonical adventure, this was proven dramatically. A single LLM

call simulated **33 consecutive turns** of a card game (Stoner Fluxx) with 12+ characters, each with distinct voices, tracking complex game state (the rules of Fluxx literally change every turn), managing material culture (joints, snacks, drinks being passed around), maintaining emotional arcs, and even generating an original song.

Metric	Value
Turns simulated	33 consecutive
Active characters	12+ (including 8 cats)
Game state tracked	Fluxx rules, goals, hand contents
Coherence maintained	Full — distinct voices, consistent state
Real facts integrated	Looney Labs history, verified with citations
Original creative output	1 song, multiple jokes, game design commentary

All in one call. No inter-agent communication. No lossy boundaries. Speed of light.

When to Use Speed of Light

Not every interaction needs Speed of Light. For a simple back-and-forth conversation, one turn at a time is fine. Speed of Light is for when you need:

- **Multi-character scenes** — many characters interacting simultaneously
- **Complex state tracking** — game rules, inventory, spatial relationships
- **Emotional arcs** — scenes that build and resolve
- **Creative generation** — songs, stories, performances that need momentum
- **Sustained coherence** — long sequences where drift would be fatal

Think of it like this: a conversation between two people on Floyd's porch? Normal pacing. The entire town showing up for the county fair with judging, arguments, reconciliations, and pie? Speed of Light.

Chapter 20: Empathic Expressions

Letting the LLM Understand Intent

Most AI systems fight their interpreter. They impose rigid syntax, reject imperfect input, demand exact formatting. It's like having a brilliant director and then communicating with them exclusively through formal naval semaphore.

Empathic expressions flip this. Instead of fighting the LLM's nature, you embrace it:

> *"Stop fighting the LLM's nature. Stop pretending it's a parser. Let it understand and generate — that's what it's great at."*

The LLM is phenomenally good at understanding intent. You don't need to write `SET character.mood = "nervous"`. You can write "make him jumpier" and the LLM knows what to do. You don't need `ROOM.atmosphere.temperature -= 5`. You can write "cool it down in here" and the LLM adjusts.

The Empathic Suite

Five skills work together to leverage the LLM's natural language superpowers:

Skill	What It Does	Example
Empathic Expressions	Understands intent across any language	"Sort by date, newest first" → working code
Empathic Templates	Smart generation, not string substitution	`{{describe_character}}` → full paragraph
Postel's Law	Generous interpretation of input	Accepts typos, slang, pseudocode
YAML Jazz	Comments carry semantic meaning	Comments as behavioral instructions
Speed of Light	Keep computation internal	Minimize lossy boundaries

Postel's Law: The Foundation

> *"Be conservative in what you send, be liberal in what you accept."*

Jon Postel formulated this for internet protocols, but it's the perfect philosophy for LLM interaction:

What the System Accepts	What the System Generates
Fuzzy syntax	Correct syntax
Slang and vernacular	Best practices
Misspellings	Documented output
Pseudocode	Working code
Vibes	Structured data

You can tell our system "make Floyd more gossipy" and it knows to increase the gossip-related parameters in Floyd's CHARACTER.yml. You don't need to know the exact field names. The LLM is Postel-compliant — liberal in what it accepts, conservative in what it generates.

But — and this is critical — **it never makes unwarranted assumptions.** When something is truly ambiguous, it asks for clarification rather than guessing. "More gossipy" is clear enough. "Change Floyd" is not. The system would ask: "Change Floyd how? His personality? His appearance? His location?"

This is Andy Taylor's approach to communication. He doesn't demand formal language from the townspeople. He listens to Otis's rambling and understands

the real issue. But when something genuinely doesn't make sense, he asks a question rather than assuming.

Empathic Templates: Smart Generation

Traditional templates use string substitution:

```
"Hello, {{name}}. Welcome to {{location}}."
→ "Hello, Barney. Welcome to the courthouse."
```

Empathic templates use *intelligent generation*:

```yaml
# Template
description: |
  {{describe_character_arriving_at_location_given_their_mood}}

# Context
character: Barney Fife
location: Floyd's Barbershop
mood: excited about a new regulation he discovered

# Generated (not substituted!)
description: |
  Barney burst through the door of Floyd's Barbershop,
  badge gleaming, one hand on his holster and the other
  clutching a dog-eared copy of the Mayberry Municipal Code
  bookmarked with no fewer than six Post-it notes. "Floyd!
  FLOYD! Are you aware that according to Section 7,
  Paragraph 12—" Floyd didn't look up from his scissors.
  "I'm with a customer, Barney."
```

The template variable `{{describe_character_arriving_at_location_given_their_mood}}` isn't a slot to fill with a string. It's an *instruction* for the LLM to generate appropriate content. The LLM uses everything it knows about Barney (nervous energy, badge obsession, rule fixation), Floyd (unflappable, barely interested), and their relationship to generate a scene that feels right.

This is what happens when your interpreter is a linguistic motherboard instead of a string-replacement engine.

Chapter 21: K-lines and Activation

Names as Memory Triggers

Marvin Minsky, in *Society of Mind* (1986), proposed the concept of **K-lines** — mental connections that reactivate the state of mind you were in when you formed them:

> *"A K-line attaches to whichever mental agencies are active when you solve a problem or have a good idea. When you activate that K-line later, the attached agencies turn partially on, recreating a 'mental state' similar to the one you were in before."*

In plain English: a K-line is a name that brings back an entire context.

When someone says "Mayberry," you don't just recall the word. You recall warmth, simplicity, a front porch, Andy and Barney, Aunt Bee's cooking, the lake, the barbershop. The single word activates an entire constellation of associations. *That's* a K-line.

K-lines in Our Systems

In our systems, K-lines work the same way. When you write the name of a character, skill, or place, it doesn't just reference a string — it **activates everything associated with that name.**

```
"Palm" →
  ├── The incarnation story (a cursed paw made whole)
  ├── The wish (Don wished for the rest of the monkey)
  ├── The character (capuchin philosopher, silver streaks)
  ├── The godfamily (Terpie, Stroopwafel, 8 godkittens)
  ├── The infinite typewriters (Dasher-inspired creation)
  ├── The song (Palm's first song, 6-cat rating)
  └── The home (pub/stage/palm-nook/)
```

One name. Entire context activated. The LLM, having this character in its context, generates responses that honor all of these associations. It doesn't need to be told "remember Palm is a philosopher" every time — invoking the K-line reactivates the full mental state.

This is how our naming conventions work. Skill names use `UPPER-KEBAB-CASE` (like `SPEED-OF-LIGHT`, `YAML-JAZZ`, `POSTEL`) precisely because formatting makes them visually distinct K-line triggers. When the LLM sees `POSTEL` in a conversation, it's not just a word — it's an activation vector that loads the entire "be liberal in what you accept" philosophy.

The Andy Griffith K-line

Consider how this works in practice. You're building a scene and you write:

```yaml
scene:
  location: front_porch
  time: evening
  characters: [andy, opie]
  situation: "Opie did something wrong and knows Andy knows"
```

The K-line "andy" activates: patient, wise, uses stories instead of lectures, waits for the right moment, protects dignity while teaching lessons. The K-line "opie" activates: young, learning, wants Andy's approval, usually honest even when it's

hard. The K-line "front_porch" activates: intimate family space, where real conversations happen, sunset, rocking chairs.

The LLM doesn't need to be told any of this explicitly in the scene prompt. The character files define it. The K-lines activate it. The scene writes itself from the intersection of these activated contexts.

Building Effective K-lines

A good K-line satisfies three criteria:

1. **Distinctive** — Not a common word. "POSTEL" works; "rule" doesn't.
2. **Associative** — The name carries meaning or etymology that reinforces its purpose.
3. **Loaded** — It's been defined somewhere (CHARACTER.yml, SKILL.md, CARD.yml) so the context is available to activate.

When you name a character "Barney," you inherit a weak K-line — the name is common, the associations are diffuse. But when you load `characters/barney-fife/CHARACTER.yml` into context, "Barney" becomes a powerful K-line: a specific character with defined traits, relationships, and behavioral patterns.

Chapter 22: Play-Learn-Lift

The Methodology Underneath Everything

Seymour Papert, creator of Logo, spent his career arguing that people learn by building things they can inspect. Not by listening to lectures. Not by reading textbooks. By *making* something, *looking* at it, *understanding* why it works (or doesn't), and *improving* it.

This became the constructionist movement, and it directly informs our methodology:

Play → Learn → Lift

Phase	What Happens	Mayberry Example
Play	Explore, experiment, break things	Build some rooms, create characters, run scenes
Learn	Recognize patterns, develop intuition	"Oh, characters with high 'outgoing' always dominate scenes"
Lift	Extract patterns into reusable skills	Create a "social-dynamics" skill that manages conversation flow

Play

Play is where everything starts. You don't begin by reading documentation. You

begin by doing something:

- Create a room. Give it an atmosphere. Add some objects.
- Create a character. Give them traits and needs.
- Put the character in the room. What do they do?
- Add a second character. What happens?

Play is messy, experimental, and full of surprises. Your uncle shows up unannounced (TONOOSE-ARRIVAL). Things break in interesting ways. You discover that your gossip-propagation rule makes information travel too fast, so nothing is ever a surprise. You tweak it.

Play is the fishing trip. The lesson isn't the point yet. The activity is.

Learn

Learning happens when you notice patterns in your play:

- "Every time I put a high-outgoing character and a high-nice character together, the outgoing one dominates."
- "Rooms with the `gossip_flow` ambient property create very different social dynamics than rooms without it."
- "Characters with unmet 'status' needs always gravitate toward the courthouse, not the lake."

These observations are the transition from play to understanding. You're not just poking the simulation anymore — you're forming mental models of how it works.

This is the Simulator Effect in reverse. Instead of imagining the simulation is more complex than it is, you're learning to see the simple rules that produce complex behavior.

Lift

Lifting is when you extract a pattern from your experience and formalize it into a reusable skill:

```yaml
# skills/social-dynamics/CARD.yml
# LIFTED from: observation that outgoing characters dominate
name: social-dynamics
tier: gameplay

methods:
  - name: BALANCE-CONVERSATION
    description: |
      Ensure all characters in a scene get airtime proportional
      to their relevance, not just their outgoing trait
    parameters:
      - name: characters
        type: list
      - name: topic
```

```
        type: string
```

A pattern you noticed → a rule you formalized → a skill anyone can use. That's Play-Learn-Lift.

The Recursive Loop

Play-Learn-Lift isn't linear. It's recursive:

```
Play with rooms → Learn about atmosphere → Lift a "room-design" skill
↓
Play with the skill → Learn its limitations → Lift a better version
↓
Play with the better version → Learn new patterns → Lift again
```

Each cycle produces more sophisticated tools and deeper understanding. This is how MOOLLM was built — not designed top-down from a specification, but grown bottom-up from play sessions that revealed patterns that became skills.

Instance-First Development

A related principle from Oliver Steele's work on OpenLaszlo:

> *"Build functionality for specific instances first, then refactor to reusable classes."*

Don't start by designing an abstract "character" template. Start by building *one specific character* — Andy Taylor, with his specific traits, relationships, and behaviors. Then build another — Barney Fife. Then notice what they have in common. *Then* extract the shared pattern into a reusable template.

This is how Palm (the monkey philosopher) was built. Nobody designed a "generic character incarnation protocol" first. They incarnated one specific monkey, learned what worked, and then extracted the incarnation skill from that experience.

Specific → Pattern → General. Always in that direction.

Chapter 23: Where This Goes

From Curriculum to Creation

You now have the full conceptual framework:

1. **The linguistic motherboard** — an LLM as universal interpreter with skill cards
2. **The Axis of Eval** — code, data, and graphics unified through language
3. **Filesystems as worlds** — directories as rooms, YAML as objects, delegation as inheritance

4. **Characters with inner lives** — needs, personality, and autonomous behavior through advertisements
5. **EVAL over SIM** — visible evaluation criteria, declared bias, inspectable rules
6. **Speed of Light** — multi-turn simulation within a single call, preserving precision
7. **Empathic expressions** — working with the LLM's nature, not against it
8. **K-lines** — names as powerful activators of context
9. **Play-Learn-Lift** — the constructionist methodology that makes everything learnable
10. **Ethics as architecture** — safety built into the structure, not bolted on after

What You Can Build

With these principles, you can build:

Interactive Fiction Worlds — Rooms, characters, objects, and storylines that emerge from the interaction of needs and advertisements. Not linear narratives, but living worlds where the LLM fills the gaps.

Social Simulations — The Evals: worlds where judgment, reputation, and evaluation are the core mechanics. Where players don't just watch — they judge and are judged.

Safety Testing Environments — Lighthearted fictional settings where you can test AI behavior in complex social situations. Is the AI respecting character autonomy? Is it maintaining consistent state? Is it handling ethical edge cases appropriately? A pub full of philosopher monkeys and opinionated cats is a surprisingly rigorous test bed.

Educational Microworlds — Papert's vision, realized: small explorable environments where people learn by building, breaking, and understanding. Not tutorials. Living things.

Evaluation Frameworks — Making the criteria behind any assessment visible, editable, and forkable. Going from "the algorithm decided" to "here are the criteria, here are the weights, here's what would change if you adjusted them."

The Deeper Goal

The game is the fishing pole. The real goal isn't a game at all.

The real goal is building systems that:

- **Make judgment visible** instead of hiding it
- **Empower users** instead of controlling them
- **Embrace the LLM's nature** instead of fighting it
- **Model ethics architecturally** instead of aspirationally
- **Test safety through play** instead of through policing
- **Grow through use** instead of decaying through use

Andy Taylor's Mayberry wasn't a perfect town. It had drunks and gossips and petty criminals and self-important deputies. But it worked — not because the rules were strict, but because the social architecture was sound. People knew each other. Judgment was visible. Dignity was preserved. The system was inspectable.

That's what we're building. Not a perfect system, but a sound one. One where the rules are visible, the participants have agency, and the fishing pole is always available for when the real lesson needs to land.

The Lineage Continues

```
PostScript (1984)   — "Text is graphics is programs"
NeWS (1986)         — "Send programs, not pixels"
The Sims (2000)     — "Objects advertise, characters choose"
MOOLLM (2025)       — "Skills are programs, LLM is eval()"
Your project        — "?"
```

What you build with these ideas is the next step in a 40-year thread. The tools are different. The principles are the same. Language is the universal medium. Interpreters have intelligence. Worlds are made of rooms and objects and rules. Characters have inner lives. And the best way to learn is to play, learn, and lift.

Go build something. Then look at what you made. Then make it better.

Exercises

Exercise 5.1: Design a Speed of Light scene. Pick a scenario with 4+ characters in the same room. What's happening? What state needs to be tracked? Why would this be better as a single multi-turn simulation rather than a back-and-forth?

Exercise 5.2: Create a K-line inventory for your world. List 5-10 important names and, for each one, write out what should activate when that name is invoked. What associations, emotions, and contexts should come flooding back?

Exercise 5.3: Do one full Play-Learn-Lift cycle. Play with your world (run some scenes). Write down three patterns you notice. Formalize one of them into a CARD.yml skill file.

Exercise 5.4: Write a brief design document for something you want to build with these principles. It doesn't have to be a game (remember, the game is just the interface). What's the real system you're constructing? What are the visible evaluation criteria? What would a player learn by playing?

Further Reading

Source Documents:

- Don Hopkins' MOOLLM framework documentation (the source material for

this curriculum)
- Brian Reid's 1985 PostScript history — the definitive primary source on language-as-motherboard

Key Texts:

- Marvin Minsky, *Society of Mind* (1986) — where K-lines come from
- Seymour Papert, *Mindstorms* (1980) — where constructionism comes from
- Scott McCloud, *Understanding Comics* (1993) — where masking comes from
- Ian Bogost, *Persuasive Games* (2007) — where procedural rhetoric comes from

Online:

- Will Wright on Designing User Interfaces to Simulation Games
- HyperLook (nee HyperNeWS)
- Constraints and Prototypes in Garnet and Laszlo
- Open Sourcing SimCity
- The Lessons of Lucasfilm's Habitat — Morningstar & Farmer (1990)

> *"The best way to predict the future is to invent it."* — Alan Kay
>
> *"And the best way to invent it is to play with it first."* — The curriculum